DBMS

Database Management Systems — Complete Study Notes

Data & Information · File System vs DBMS · Architectures · Schemas & Data Independence · Data Models · ER Model · Keys · Cardinality · Codd’s 12 Rules · Normalization · Functional Dependency · SQL · Transactions · Concurrency Control · NoSQL · ACID

Typed from the handwritten class notes — every topic, example and diagram retained, with extra clarifications where the page was ambiguous.

Contents

1. Data, Information and Database 4

1.1 Data . . . 4

1.2 Information . . . 4

1.3 Database . . . 4

1.4 DBMS (Database Management System) . . . 4

2. File System and its Difference from DBMS 5

2.1 File system . . . 5

2.2 Difference between File System and DBMS . . . 5

2.3 Data access cost (file system / DBMS) . . . 6

2.4 Concurrency level . . . 6

2.5 Different level of access . . . 6

2.6 VIEW = Virtual Table . . . 6

3. DBMS Architecture 6

3.1 Two-tier / two-level architecture . . . 6

3.2 Three-tier / three-level architecture . . . 7

4. Three-Level Abstraction / Three-Schema Architecture 7

(1) External schema / View level . . . 8

(2) Conceptual schema / Logical level . . . 8

(3) Physical schema / Internal schema . . . 8

Data independence (written as “Data Dependence” in the notes) . . . 8

5. Basic Terms and Data Models 9

5.1 Some points (units of data) . . . 9

5.2 Data model . . . 9

Data dictionary . . . 11

ER diagram symbols . . . 11

Types of attributes . . . 12

More data models (numbered 12–14 in the notes) . . . 12

6. Keys in DBMS 13

Types of keys . . . 13

Master capsule — counting super keys (n = number of attributes in the relation) . . . 14

7. Mapping Cardinality / Degree of Relationship 15

8. Codd’s Rules (total 12, plus Rule 0 for the base) 16

9. Normalization 17

Types of normalization . . . 18

10. Functional Dependency (FD) 20

Types of FD . . . 21

11. SQL Commands 21

Working of commands . . . 22

Data types . . . 23

Structure of an SQL command . . . 23

SQL logical operators . . . 23

Commands with syntax and examples . . . 24

Some queries . . . 25

SQL functions . . . 25

12. SQL Constraints 25

13. SQL JOIN 27

14. Serializability 28

Types . . . 28

15. Two-Phase Locking (2PL) 28

Advantages of 2-phase locking . . . 29

Variations of two-phase locking . . . 29

16. Timestamp Concurrency Control 29

17. NoSQL 29

Types of NoSQL databases . . . 30

Comparison of NoSQL types . . . 31

18. ACID Properties 31

Appendix — Clarifications made while typing the notes 31

1. Data, Information and Database

1.1 Data

Data — raw facts or figures that are in an unorganised form and do not have any proper meaning are called data. e.g. text, number, audio ... etc.

1.2 Information

Information — when various operations are performed on data and it becomes meaningfully processed. Ex: a table.

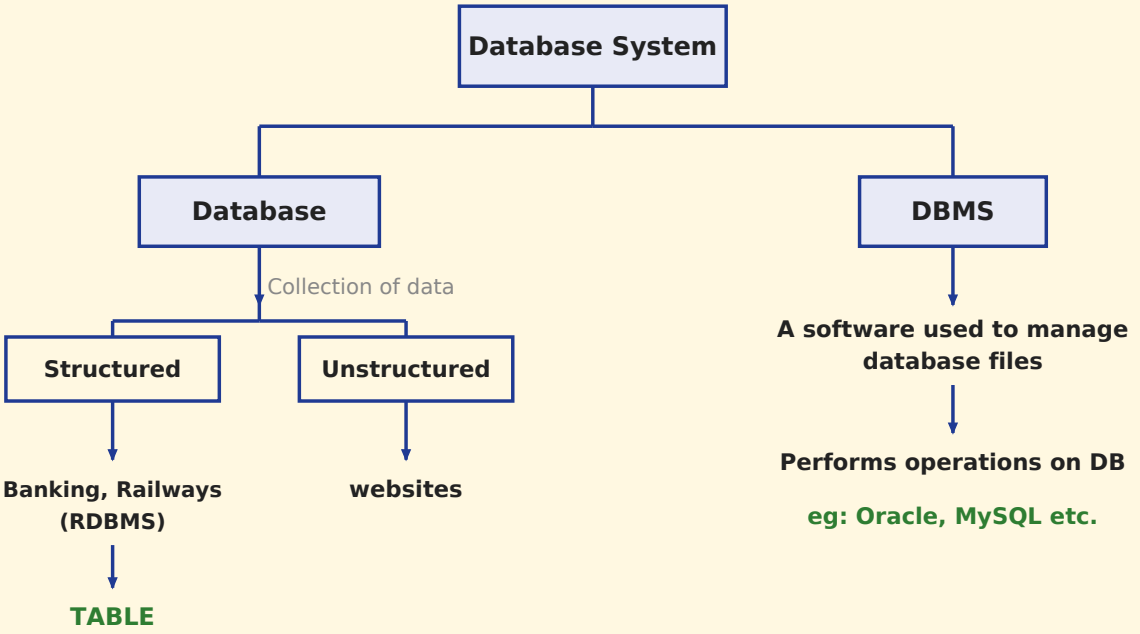


Data → Process → Information → Knowledge → Wisdom

1.3 Database

- A **database** is a connection (collection) of inter-related data.
- Types: **Structured** and **unstructured**.

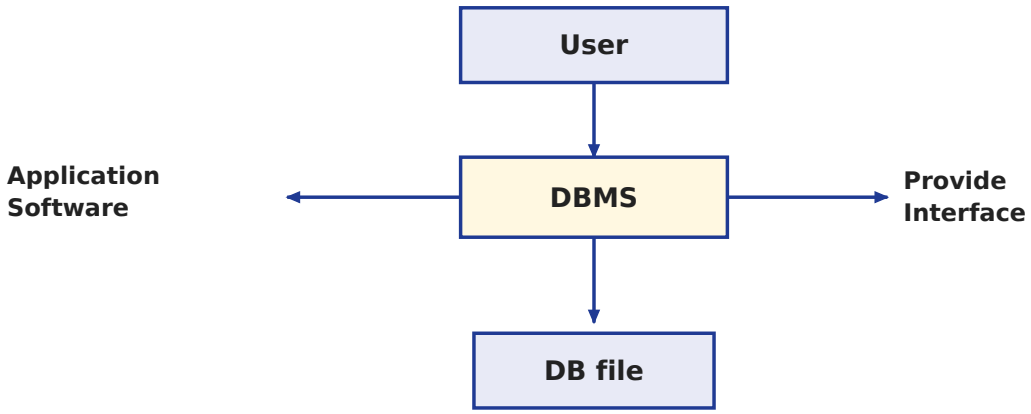
Example: Database System



	Structured database	Unstructured database
Examples	Banking, Railways (RDBMS)	Websites
Stored as	TABLE	—

1.4 DBMS (Database Management System)

- A **software used to manage database files**; it performs operations on the DB. e.g. Oracle, MySQL etc.
- It acts as an **interface between the user and the DB file**.
- It converts the user's request into a database query, sends it to the DB file, fetches the required data, and returns the information to the user.
- It is a **collection of application software** that enables communication between the user and the DB file.



User → DBMS (application software, provides interface) → DB file

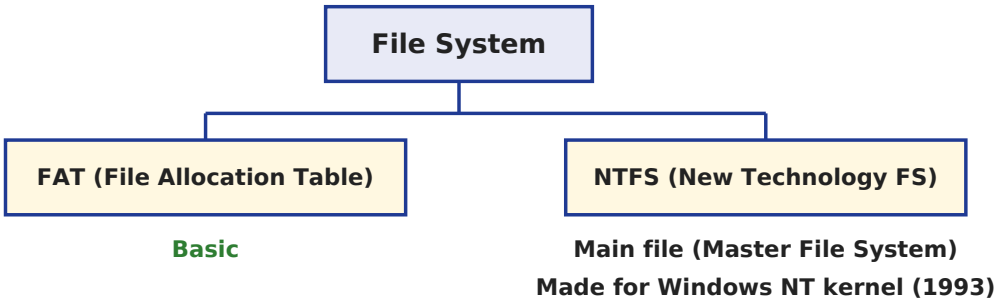
2. File System and its Difference from DBMS

2.1 File system

- To manage various files in a computer; managed by the OS.
- 2 types:

Type	Details
FAT (File Allocation Table)	Basic file system.
NTFS (New Technology File System)	Main file (Master File System); made for the Windows NT kernel (1993).

- Used for small files.



Types of file system.

2.2 Difference between File System and DBMS

#	Point	File System	DBMS
1	**Data redundancy**	Same data gets repeated / duplicated in many files.	No duplication.
2	**Data inconsistency**	If data in one file is updated, other files are not automatically updated, so data becomes inconsistent.	Data remains consistent and accurate across.
3	**Security**	Hard to give different users different types of access.	Advanced security; can give multiple users different access permissions.
4	**Concurrency**	Multiple users cannot reliably access the same file at the same time.	Multiple users can access the same data / file at the same time safely.

- **In DBMS**, a user can access data without knowing all the internal details. The user is given only the information needed (a specific row, table or column).

2.3 Data access cost (file system / DBMS)

- In a file system, bringing data from **secondary memory to primary memory is costly** because the entire file must be opened.
- In a DBMS, **indexing** is used, so the cost is low.

2.4 Concurrency level

File System	DBMS
Concurrency is at the **file / resource level**. When one user is using a file, another user cannot use (access / modify) that same file at the same time.	Concurrency is at the **row / record level**. When one user is using a particular row, only that row is locked for other users; all other rows remain available for everyone.

2.5 Different level of access

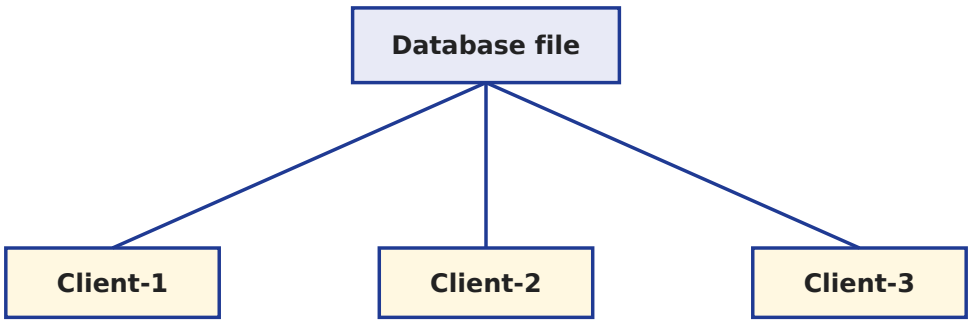
- **File system:** to give different access, separate physical files are needed.
- **DBMS:** different **views** are created.

2.6 VIEW = Virtual Table

- A view performs operations on the underlying master table.
- It does **not** physically store data.
- It only **looks like** the original physical table.

3. DBMS Architecture

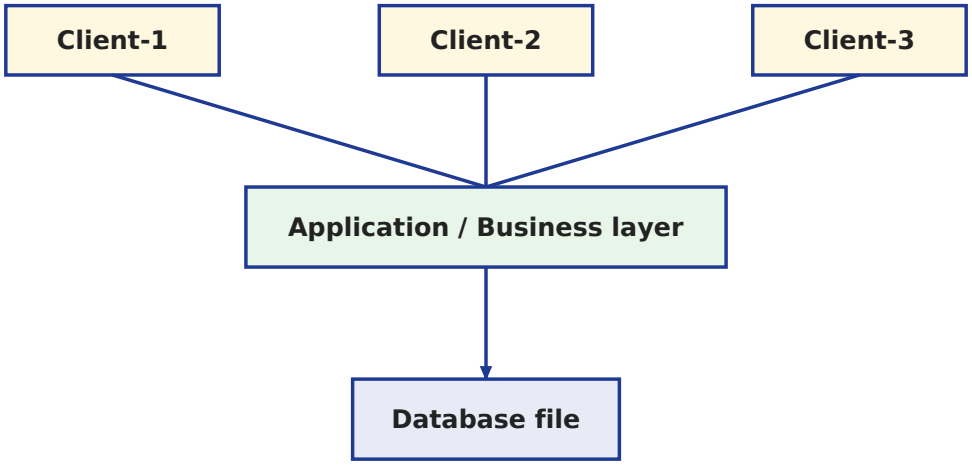
3.1 Two-tier / two-level architecture



Two-tier architecture: every client talks directly to the database file / server.

- The client connects **directly** to the database server.
- **Client–server** architecture.
- Limited number of users.
- Less secure, easy to maintain.
- The process on the client is converted into a database query (often called a low-level language query) and sent to the server.

3.2 Three-tier / three-level architecture

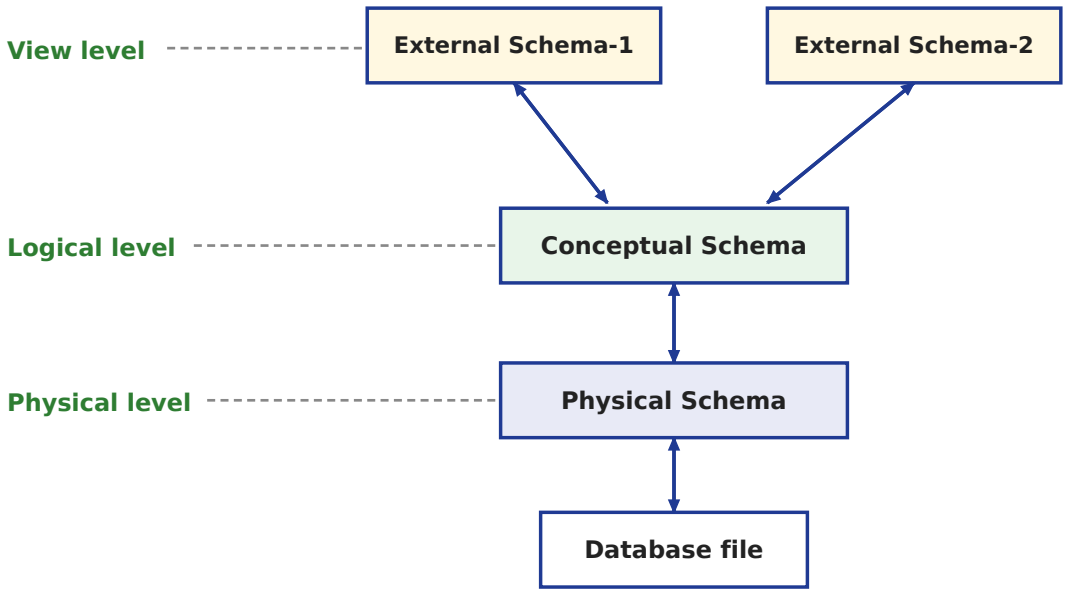


Three-tier architecture: clients → application / business layer → database file.

- The user / client is **not directly connected** to the DB server.
- The client's request is first sent to the **application / business layer**.
- The application layer then converts the request to a low-level database query and interacts with the DB server.
- The application layer acts as an **interface between client and server**.
- Because clients do not talk to the server directly, this architecture provides **better security**.
- Most web apps use it (e.g. Facebook, Instagram, Amazon).

	Two-tier	Three-tier
Client connects to	DB server directly	Application / business layer
Users	Limited number	Many (web scale)
Security	Less secure	Better security
Maintenance	Easy to maintain	—

4. Three-Level Abstraction / Three-Schema Architecture



View level (external) ↔ logical level (conceptual) ↔ physical level (internal) ↔ database file.

(1) External schema / View level

- Users are connected to the DB.
- Each user gets a **different view / schema** according to need.
- Conceptual and physical details are **hidden** from the user.
- Which table actually stores the user's data, how that table is related to other tables and on which disk the data is stored — all hidden.

(2) Conceptual schema / Logical level

- Describes **what data is stored** in the database.
- The overall design of the DB is made using this schema.
- Based on this, tables are created and the relationships between tables are defined / created.

(3) Physical schema / Internal schema

- Shows **how the actual database files are stored** on storage devices.
- Managed by the **DBA**.
- We work on the actual disk; it controls how data is placed on and retrieved from the disk.
- Maintaining the **data dictionary**.
- Controls the internal working of the database.

Summary (cloud in the notes)

Physical schema = deals in storage of DB
Conceptual schema = deals in tables and their relationship
External schema = deals in requirement of user

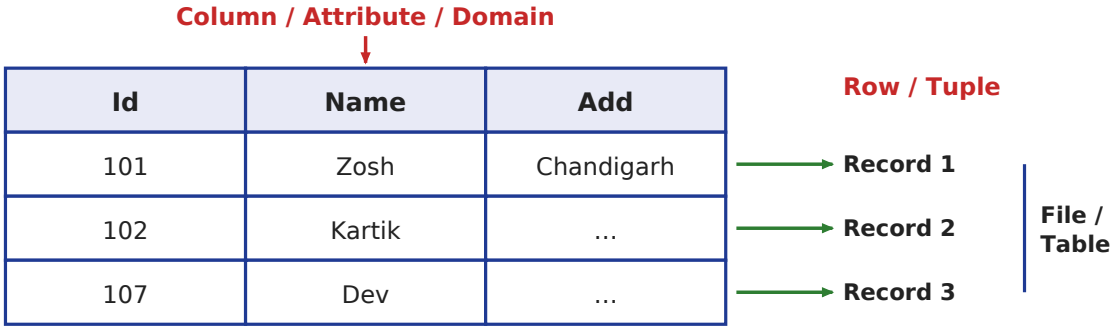
Data independence (written as "Data Dependence" in the notes)

	Logical data independence	Physical data independence
Meaning	If we add a new column to a table or change relationships between tables, the end-user interface (view) does not change.	If we add new storage / disk or change the disk location of data, the tables and their relationships are not affected.
Independence between	View level ↔ Conceptual level	Conceptual level ↔ Physical level (storage)

5. Basic Terms and Data Models

5.1 Some points (units of data)

#	Term	Meaning
1	Bit	Smallest unit to represent data
2	Character	Collection of bits
3	Field	Collection of characters
4	Record	Collection of fields
5	File	Collection of records
6	Database	Collection of fields (as written in the notes; strictly a collection of related files / tables)



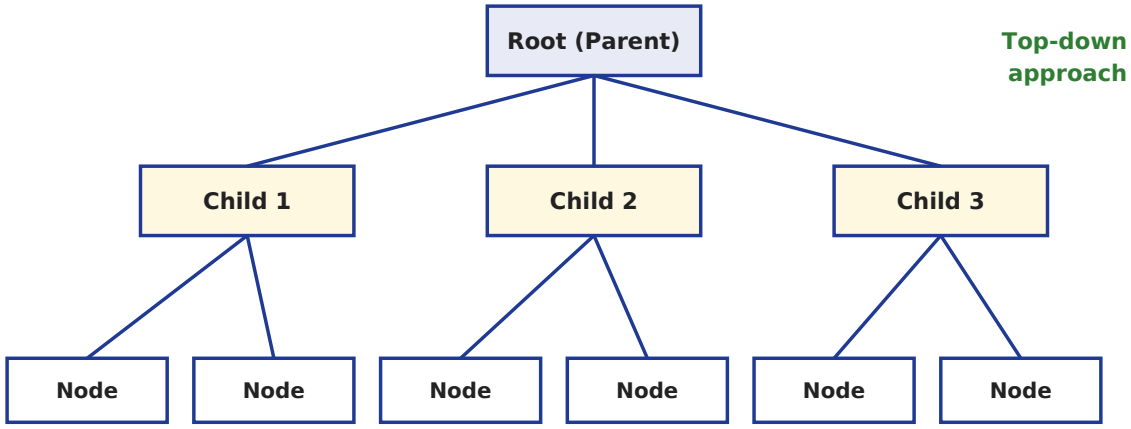
A table: columns (attribute / domain), rows (tuple / record), the whole table = file / table. (Sample values as in the notes.)

5.2 Data model

- **Data model** = data representation in a structural way.
- Types:

(1) Hierarchical data model

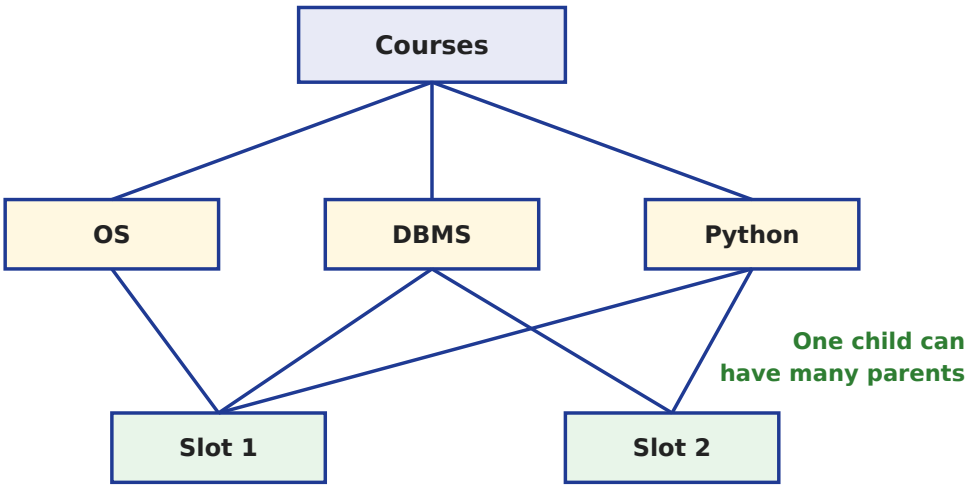
- Developed by **IBM (1968)**; developed for **IMS** (Information Management System).
- Represents **parent-child / one-to-many** relationship.
- **Tree** data model.
- One child of a parent cannot see another child's information.
- **Single-parent** model.
- Each record is called a **node / segment**.
- **Top-down** approach.



Hierarchical model: a tree — each child has exactly one parent.

(2) Network data model

- Extension of the hierarchical data model.
- Based on parent–child relationship.
- **One child can have more than one parent.**
- Uses **graph** data structure.
- **Many-to-many** relationships.
- Data elements are **nodes** and relationships are **edges** in the graph.



Network model example: Courses → OS / DBMS / Python → Slot 1 / Slot 2.

(3) Relational data model

- Considered the **best** data model because data is represented in table format.
- A table = row + column.
- A table is called a **relation**.
- A column represents a property of one entity.
- Each record in the table should not be repeated.
- A row (record / tuple) represents an entity.
- This model follows **E.F. Codd's first rule**: data must be arranged in the form of rows and columns in tables.
- E.F. Codd defined **12 rules** for the relational model.

(4) Entity–Relationship (ER) model

(A) Entity

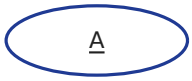
- Any object that has a physical appearance or exists physically is called an **entity**.
- e.g. in a student table, attributes like Rollno, Name describe the **student entity set**. Each individual student is an **entity**.
- In this data model, the database is **represented graphically**, so the complete logical structure of the database is shown.
- Using this model, the **schema** of the database is represented theoretically.
- The logical representation of data = **schema**. e.g. **Student (Id, Name, Class, Phone)** ← relation.
- A schema shows things like table, primary key, foreign key, view.
- The **instance** of a DB is the **snapshot** of the data available at a particular moment of time.

(B) Relation → connection of 2 entities. **(C) Attributes** → characteristics of an entity.

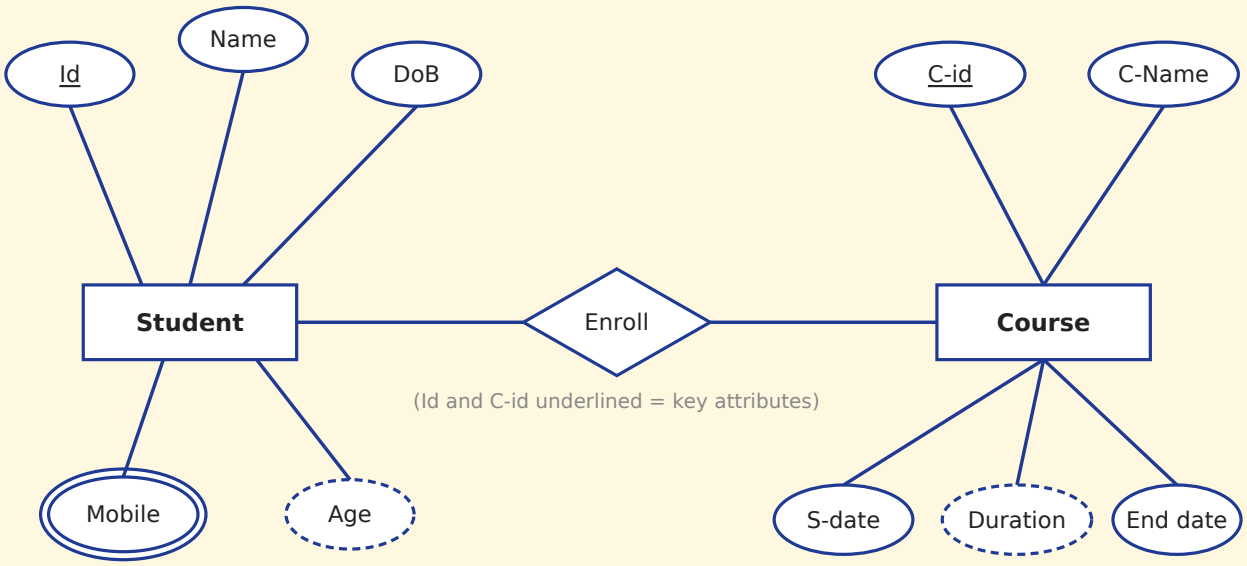
Data dictionary

- Data about data = **metadata**.
- Stores additional information about data, including storage and physical details.
- Managed by the **DBMS itself**.
- Stored in the DB's **table space** (the internal storage area where all the tables are stored).

ER diagram symbols

Symbol	Shape	Meaning
	Rectangle	Entity
	Ellipse	Attribute
	Diamond	Relationship
	Lines	Entity ↔ relationship, entity ↔ attributes
	Double ellipse	Multivalued attribute
	Dashed ellipse	Derived attribute
	Double rectangle	Weak entity
	Double diamond	Weak-entity relationship
	Ellipse with line (underlined)	Key attribute

Example: ER diagram — Student enrolls in Course



Student (Id, Name, DoB, *Mobile* — multivalued, *Age* — derived) — **Enroll** — **Course** (C-id, C-Name, S-date, Duration, End date). Mobile is a double ellipse (multivalued); Age is a dashed ellipse (derived from DoB).

Note

In the handwritten diagram it is hard to tell which of the Course attributes are dashed. Duration (= end date − start date) is drawn as a derived attribute here; check it against your own page.

Types of attributes

#	Type	Meaning	Example
1	**Single-valued**	Carries a single value	Roll no.
2	**Multi-valued**	Carries multiple values	Mobile no.
3	**Simple**	Cannot be divided (carries a single value)	Registration no.
4	**Composite**	Made up of sub-parts	Name (F-name, Last-name)
5	**Stored**	Attribute given to (stored in) the table	—
6	**Derived**	Is not kept in the table; calculated from other data	Age
7	**Key**	To identify a record uniquely	Primary key
8	**Non-key**	Does not uniquely identify the record	Name, DoB
9	**Complex**	Multivalued + composite	Address
10	**Required**	Compulsory attribute (NOT NULL)	—
11	**Optional**	Not compulsory	—

More data models (numbered 12–14 in the notes)

(5) Object-oriented data model

- Data is stored in the form of **objects**.
- Values are stored in objects instead of variables.
- Used in high-level languages.
- Can support complex data types and **inheritance**.

(6) Document data model

- Subset of other data models; **NoSQL** (no SQL use).
- Used to map objects in programming.

(7) Key-value data model

- **NoSQL** (no SQL use).
- **Simple and fast**.

6. Keys in DBMS

- Keys **uniquely identify** each record / table.
- According to **E.F. Codd's Rule 1**, two records of a table cannot be identical.
- A key can be formed from **one or more attributes**.

Types of keys

(1) Candidate key

- The **minimum set of attributes** that can uniquely identify a table / record is called a candidate key.
- Columns Sid, Address, Name: if Sid uniquely identifies each record, then Sid is a candidate key.
- A table can have multiple unique keys; **one is chosen as primary key** and the remaining are still candidate keys.
- Values of a candidate key must always be **unique and NOT NULL**.
- A candidate key can be **simple or composite**.

Example: Simple vs composite candidate keys

Candidate keys (A, BC, E) → **A, E** are simple; **BC** is composite.

(2) Primary key

- Must be **unique and not null**.
- A primary key is also a candidate key whose values are always unique and not null.
- A table can have **only one** primary key.

Example: Student (Sid, Name, Class, Phone)

Sid = primary key (also a candidate key). Another unique column such as Phone, if present, would be a candidate key that was not chosen → **alternate key**.

(3) Alternate key

- All candidate keys **except the chosen primary key**.
- May be allowed to be null (as written in the notes).
- Also called **secondary key**.

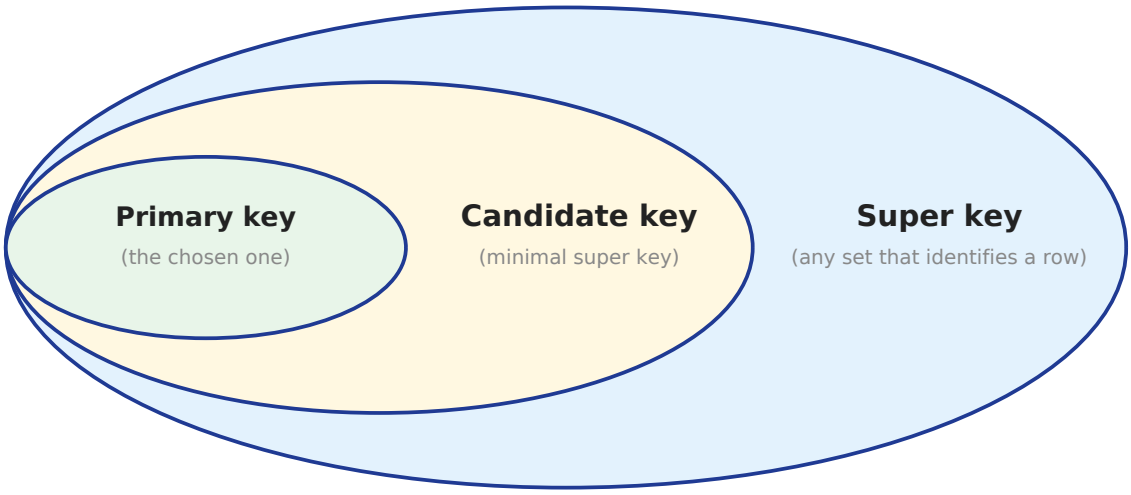
Prime / key attributes

Example: Relation R (A, B, C, D, E)

Candidate keys: **(A, BC, E)** → prime attributes: **A, B, C, E**. Then **D** becomes a **non-prime (non-key) attribute**.

(4) Super key

- A set of attributes that can uniquely identify a table / record.
- (Sid, Name), (Name, F_name), (Sid, Class) etc. are super keys.
- Every candidate key is a super key, but not every super key is a candidate key (it may contain extra, unnecessary attributes).
- Must contain at least one attribute of a candidate key, but it is not necessary that all its attributes are from the candidate key.
- The **minimum super keys are called candidate keys**.



Primary key ⊂ Candidate key ⊂ Super key.

Master capsule — counting super keys (n = number of attributes in the relation)

Case	Number of super keys
1) Only one candidate key, a single attribute (e.g. A)	2^{n-1}
2) Two candidate keys, each a single attribute (e.g. A and B)	$2^{n-1} + 2^{n-1} - 2^{n-2}$
3) One candidate key made of two attributes (e.g. AB)	2^{n-2}

Example: Question 1: R (A, B, C, D) and CK = A

n = 4. Super keys = $2^{n-1} = 2^3 = \mathbf{8}$.

Example: Question 2: R (A, B, C, D) and CK = A and B

Super keys = $2^{n-1} + 2^{n-1} - 2^{n-2} = 2^3 + 2^3 - 2^2 = 8 + 8 - 4 = \mathbf{12}$.

Example: Question 3: R (A, B, C, D) and CK = ABCD

The only super key is ABCD itself → number of super keys = **1**.

Correction to the handwritten notes

In the handwritten Question 2 the middle sign looks like "+" ($2^3 + 2^3 + 2^2$). The formula in the Master Capsule — and the stated answer 12 — need a minus: 8 + 8 − 4 = 12.

(5) Foreign / Reference key

- The **primary key of one table is used as a column in some other table**.
- Can be **NULL and not unique**; no uniqueness constraint applies.
- The **referenced table** contains the primary key; the **referencing table** contains the foreign key.

Referenced table / parent table

Operation	Effect	Solution
(a) Insert	No violation (inserting a parent row cannot break a reference)	—
(b) Delete	May cause violation	ON DELETE SET NULL, ON DELETE CASCADE, ON DELETE NO ACTION
(c) Update	May cause violation	SET NULL in the referencing table on update, ON UPDATE CASCADE, ON UPDATE NO ACTION

Referencing table / child table

Operation	Effect
(a) Insert	May cause violation in the referenced table
(b) Delete	No violation
(c) Update	May cause violation

(6) Composite key

- A key that uses **more than one column**.

7. Mapping Cardinality / Degree of Relationship

(A) One-to-one

- The relationship table has at least two attributes, which are the **primary keys of the two tables**.
- Any attribute can be chosen as the primary key.
- Minimum two tables.

Eid	Name	Age
E1	N1	22
E2	N2	20
E3	N3	18

Employee

Eid	Did
E1	D1
E2	D2
E3	D3

Work (relationship table)

Did	Dname	Loc
D1	DN1	L1
D2	DN2	L2
D3	DN3	L3

Department

(B) One-to-many

- In this type of relationship, the relationship table uses the **primary keys of both tables as foreign keys**.
- Each employee (P.K) can appear against many projects (F.K).

Eid	Name	Dept
E1	A	IT
E2	B	ME
E3	C	Civil

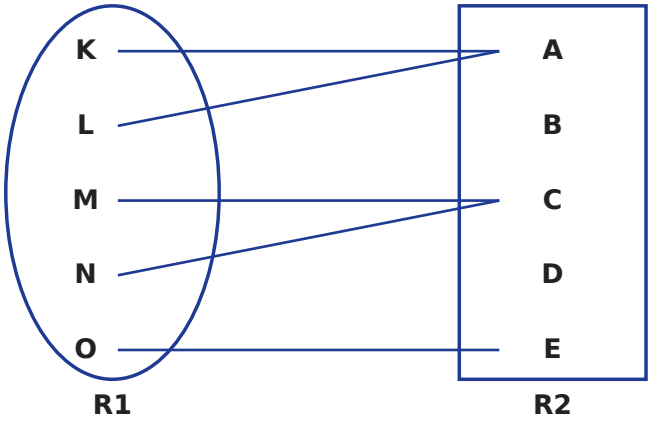
Employee (Eid = P.K)

Project	Eid (FK)	Start_Date
P1	E1	Feb
P2	E1	Mar
P3	E1	Apr
P4	E2	May
P5	E2	June
P6	E3	July

Project (Eid = F.K)

(C) Many-to-one

- Many rows of R1 point to one row of R2.



K and L → A
M and N → C
O → E

Several R1 elements share the same R2 element (B and D are not used).

Extra

Many-to-many (not in the handwritten notes): needs a separate relationship table holding the primary keys of both tables as foreign keys.

8. Codd's Rules (total 12, plus Rule 0 for the base)

(1) Rule Zero — Foundation / base rule

- For a system to be called an **RDBMS**, it must qualify as a relational database management system.
- It must use its relational capabilities to qualify as an RDBMS.

(2) Rule One — Information rule

- All information in a database must be represented in **one and only one way**, as values in a table (in the form of rows and columns).

(3) Rule Two — Guaranteed access rule

- The entire data in a database must be accessible.
- Every table must have a **primary key**.
- Data is logically reachable using the primary key.

(4) Rule Three — Systematic treatment of NULL values

- Every field in an RDBMS must be allowed to be null / empty.
- NULL values are used to represent **missing data, unknown data or not-applicable data**.

(5) Rule Four — Active online catalogue based on the relational model

- Each system must have an online relational catalog.
- It must be accessible to authorised users using the same regular query language they use for data.

(6) Rule Five — Data sublanguage rule

- Must provide at least one relational language to access data.
- The language should support linear syntax. e.g. DDL, DML.

(7) Rule Six — View updating rule

- A view must be updatable.

(8) Rule Seven — High-level insert, delete and update rule

- Must support insert, delete and update operations through a high-level relational language.
- An operation should work on a **set of rows** (a retrieval set), not only on a single row at a time.

(9) Rule Eight — Physical data independence

- When any change is made in storage / disk, it must not affect the database tables or their relationships.

(10) Rule Nine — Logical data independence

- When modifications are made at the logical level of the database, they must not affect the end-user's view.
- Achieving logical data independence is difficult.

(11) Rule Ten — Integrity independence rule

- Integrity constraints must be stored in the **database catalog**, not inside the application program.
- e.g. in a Student table the constraint "phone is NOT NULL" is defined in the DB, so any inserted record must have a non-null phone.

(12) Rule Eleven — Distribution independence rule

- If different parts of the data are distributed at various locations, this must be invisible to the DB user.

(13) Rule Twelve — Non-subversion rule

- Using any lower-level language you must not be able to ignore the integrity rules imposed by the DB language.
- Constraints enforced by the DB language cannot be bypassed.

Note

The notes number the rules (1)–(13) next to "Rule Zero … Rule Twelve"; one duplicate "Rule Seven" is crossed out. They are listed here as Rule 0 to Rule 12.

9. Normalization

- Normalization **minimises data duplicacy / redundancy** in a table.
- When a table has data redundancy, it leads to **three types of anomalies / problems** in the table:

Example: Table with redundancy (values are illustrative — the notes' sample is only partly legible)

Sid	Name	Address	Cid	CName	Teacher	Fee
S1	N1	Jaipur	C1	CN	T1	Null
S2	N2	JPR	Null	C	T2	10k
S3	N3	Patna	C2	C++	T3	15k
S4	N4	Agra	C3	DS	T4	Null
S5	N5	Ranchi	Null	Null	T3	Null

(a) Insert anomaly

- When a record is inserted into a given table, for columns like Cid, CName, Teacher, Fees we sometimes have to enter **NULL or dummy values**.
- This increases the number of null / dummy values in the DB.
- This situation is called an **insert anomaly**.

(b) Delete anomaly

- When a record is deleted from the DB, its (other related) data is also deleted.

(c) Update anomaly

- When a column in the DB is updated, more than one related field has to be (or gets) updated.

Solution

Solution of anomalies: decomposition of the table.

Types of normalization

(1) 1NF (First Normal Form)

- According to RDBMS guidelines, every table should be in **1NF**.
- Each attribute (column) of a table must hold only a **single value in each row** (no multivalued fields).

Example: Table NOT in 1NF

R.No	Name	DoB	C-Name
101	A	11/12/...	C++
102	B	17/11/...	Java
103	C	—	Py, C++, DS

C-Name has multiple values, so it is **not in 1NF**. To convert it to 1NF, three methods can be used:

① **Repeat Rollno** (repeat the row for each value — high degree of dependency). ② **Repeat column**. ③ **Split table**.

Example: Split table (method ③)

Roll	Name	DoB
101	A	11/12/...
102	B	17/11/...
103	C	—

Rollno	C-Name
101	C++
102	Java
103	Python
103	C++
103	DS

Questions on 1NF

Example: Q1 — Relation R (P, Q, R, S, T) with candidate keys {P,Q}, {P,R} and {R,S,T}. Is it in 1NF or not?

Answer: Yes, 1NF.

Reason: if some attribute is common to **all** candidate keys, that attribute is multivalued and the relation is **not** in 1NF. Here no attribute is common to all three candidate keys, so there is no multivalued attribute; hence the relation is in 1NF.

Example: Q2 — Relation P (U, V, W, X, Y) with candidate keys UV, VWX, VX, VY

V is common to all candidate keys → multivalued = **Yes**; 1NF = **No**.

Example: Q3 — Relation R (P, Q, R, S, T) with candidate keys PR, PQ, RQ, ST, PST. How many multivalued attributes are there?

Answer: 0. No attribute is common to all candidate keys, so it is also in 1NF.

(2) 2NF (Second Normal Form)

- When **a part of a candidate key (a prime attribute) alone determines a non-prime attribute**, this is called a **partial dependency**.
- Must be in 1NF.
- No partial dependency.
- All non-prime attributes must be **fully functionally dependent** on the candidate key.

Example: Q1 — Relation R1 = (A, B, C, D, E, F), FD = {C → F, E → A, EC → D, A → B}, C.K = given in the notes. Is it in 2NF?

Answer: No, it is not 2NF — because A → B where A is a **prime** attribute and B is **non-prime** (a part of a key determines a non-prime attribute).

Clarification (added)

Check: the closure of {E, C} is {A, B, C, D, E, F}, so EC is a candidate key; C → F and E → A are partial dependencies (a part of EC determining non-prime F and A). Either way the conclusion “not in 2NF” holds.

Example: Q2 — Relation R1 = (K, L, M, N, O, P), FD = {MN → P, P → L, O → N}, CK = {L, MN, O, P}

Answer: 2NF ✓ — all the dependencies lie within the circle of prime attributes (L, MN, O, P), so no non-prime attribute is partially dependent.

Note

The candidate keys of the two practice questions (2NF Q2, 3NF Q1) are copied exactly as written in the notes; the reasoning (what kind of attribute is on each side of every FD) is what the answer rests on.

(3) 3NF (Third Normal Form)

- Must be in 2NF.
- Should have **no transitive dependency**.
- When one **non-prime attribute determines another non-prime attribute**, this is called transitive dependency.

Prime → Non-prime	Non-prime → Non-prime
Partial dependency (violates 2NF)	**Transitive** dependency (violates 3NF)

Example: Q1 — Relation R1 = (K, L, M, N, O, P), FD = {K → M, MN → O, O → K, P → L}, C.K = {K, M, ON}

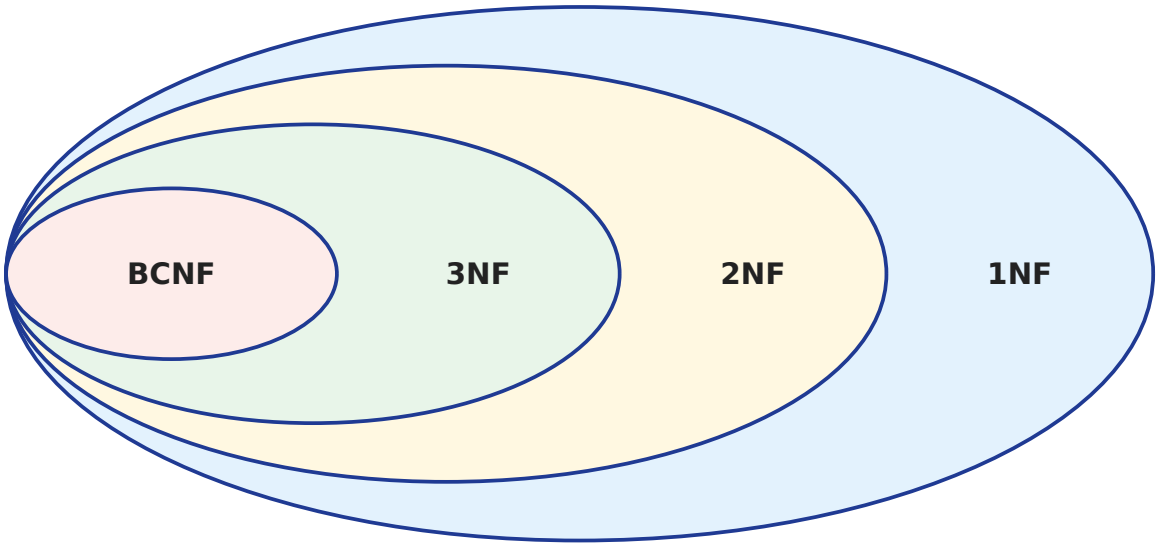
Answer: Not in 3NF. P → L is non-prime → non-prime, a transitive dependency.

(4) BCNF (Boyce–Codd Normal Form)

- Relation must be in **3NF**: atomic values, no partial dependency, no transitive dependency.
- The **left-hand side of each functional dependency must be a super key (or candidate key)**.

Example: Q — Relation R = (A, B, C, D, E, F), CK = (A, C, D, F, E), FD = {C → D, F → E, A → E, D → F}

Every left side (C, F, A, D) is part of the candidate-key set (marked “C.K” under each FD in the notes), so **Yes — BCNF**. All attributes lie within the circle.



BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF

BCNF is the strictest: BCNF ⊂ 3NF ⊂ 2NF ⊂ 1NF.

(5) 4NF (Fourth Normal Form)

- 4NF and 5NF are the **highest levels of normalization**.
- 4NF controls **multivalued dependency**.
- A multivalued dependency exists when a table has more than one independent multivalued attribute.
- For such a dependency we need at least three attributes: two independent attributes that both depend on a third attribute.
- Must be in BCNF.
- No multivalued dependency.

Example: Question — table with multivalued dependencies

SID	SName	CID	CName
S1	A	C1	C
S2	A	C2	D
S3	B	C1	C
S4	B	C2	D

Original table

SID	SName
S1	A
S2	A
S3	B
S4	B

R1

CID	CName
C1	C
C2	D

R2

Multivalued dependencies: SID ↠ CID, SID ↠ CName, SName ↠ CName. Decompose into **R1 and R2** (R1 ⋈ R2 gives back the original table).

Note

The handwritten sample table is small and slightly hard to read; the values above follow it as closely as possible.

(6) 5NF (Fifth Normal Form)

- Any table that is already in 4NF and has **no join dependencies** is in 5NF; also called **PJNF** (Project–Join Normal Form).
- **Non-loss (lossless) decomposition**:
- When we split a table to remove redundancy and later join / merge the tables back, **no data should be lost**.

Normal form	Requirement
1NF	Single (atomic) value in each column of each row
2NF	1NF + no partial dependency
3NF	2NF + no transitive dependency
BCNF	3NF + LHS of every FD is a super key / candidate key
4NF	BCNF + no multivalued dependency
5NF (PJNF)	4NF + no join dependency (lossless decomposition)

10. Functional Dependency (FD)

- The relationship between attributes of a relation is called functional dependency.

Sid	Name	Age
101	N1	22
102	N2	25
103	N3	30
104	N4	27

- From **S_id** we can uniquely identify **Name**. So: **S_id → Name**.
- S_id is the **determinant**.
- Name is the **determined / dependent** attribute.

- For a determinant: if its value is the same, the determined attribute's value will always be the same. But if the determined attribute has the same value, the determinant does not have to be the same.

Types of FD

(A) Trivial FD

- If X, Y are attributes of relation R, then X → Y is trivial if **Y is a subset of X** (X ⊇ Y).
- Every attribute determines itself: **Sid → Sid**.
- **Sid + Name → Name** (Y is a subset of X, X ≥ Y).
- The attributes on the left include all the attributes on the right, or both sides have exactly the same attributes.

Example: Let Eid + Ename → Eid + EAge. Will it be a trivial FD?

X = {Eid, Ename}, Y = {Eid, EAge}. Y is not a subset of X (X ≠ Y), so it is **not trivial** — but it is **semi-trivial**, because they share Eid.

(B) Non-trivial FD

- X → Y is non-trivial when X and Y have **nothing in common** (X ∩ Y = ∅). e.g. **Sid → S_Name**.
- In a non-trivial FD the determinant always determines a **new** attribute.

(C) Semi-trivial FD

- X → Y is semi-trivial when Y is **not a subset of X** but X and Y still share some attribute.

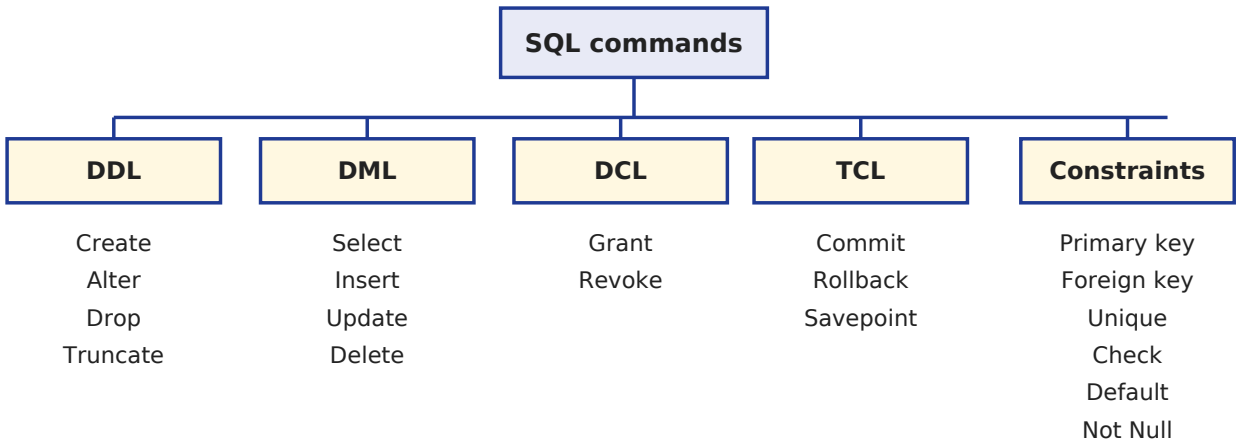
Example: Sid + Sname → Sid + SAge

Y is not a subset of X; the common part (Sid) can be dropped, so it reduces to **SName → SAge**.

Clarification

The notes write the non-trivial condition as "x ∩ y ≠ 0" with the example Sid → S_Name, whose sides are disjoint. The standard condition used here is X ∩ Y = ∅.

11. SQL Commands



Classification of SQL commands.

Group	Full form	Commands
(A) DDL	Data Definition Language	Create, Alter, Drop, Truncate / Terminate
(B) DML	Data Manipulation Language	Select, Insert, Update, Delete
(C) DCL	Data Control Language	Grant, Revoke
(D) TCL	Transaction Control Language	Commit, Roll Back, Save Point

Group	Full form	Commands
(E) Constraints	—	Primary key, Foreign key, Unique, Check, Default, Not Null

Working of commands

(1) Create

- Creation of a **database** (and tables).
- Table name — **space not allowed**; use "_".

(2) Alter

- Add a new column in a table.
- Drop / delete a column.
- Change data type.
- Change name / size.
- Change table name.
- Add a constraint.
- Update table structure.

(3) Drop

- Used to delete **both the table structure and its data** together.
- Deletes the complete table.
- A table deleted using DROP **cannot be recovered**.

(4) Truncate / Terminate

- This command deletes **all data in the table at once**.
- It does **not** delete the table structure.
- The deleted data cannot be recovered.

(5) Delete

- Used to delete records (rows) from a table.
- We use a **WHERE** clause so that only the selected data is deleted.
- Without a WHERE clause, **all data** in the table will be deleted.

(6) Insert

- Used to insert new records into a table.
- While inserting with this command, values must be given for all columns of the table.
- NULL values can also be given.

(7) Update

- To modify existing data in a table.
- With UPDATE, if a WHERE clause is used, only the selected data is updated.
- If no WHERE, **all records** in the table will be updated.

(8) Select

- Standard query by which we retrieve table data.

(9) Grant

- When we allocate some permission to a user on the DB.

(10) Revoke

- If we want to deny / deallocate the permission of the user on the DB.

(11) Commit

- The transactions / queries done by the user are stored **permanently** in the DB.

(12) Roll back

- The last (uncommitted) transactions are reverted — the DB returns to the last committed state.

(13) Save point

- Used to mark a particular point in a transaction so that we can roll back only the selected part of the transaction.

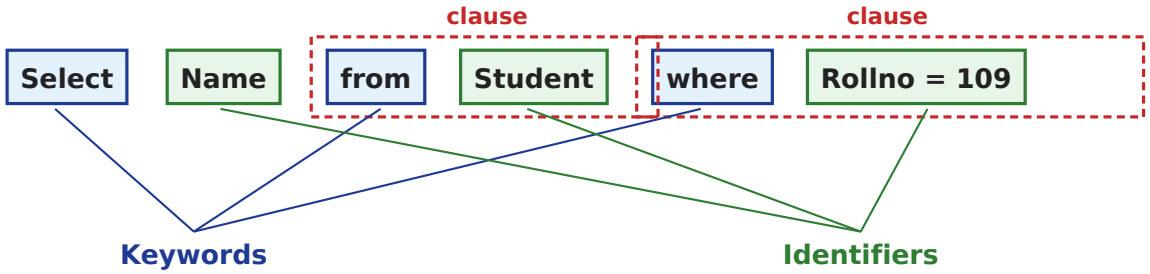
	DROP	TRUNCATE	DELETE
Removes	Structure + data	All data	Selected rows (WHERE) or all rows
Table structure	Deleted	Kept	Kept
Recoverable?	No	No	Can be rolled back before commit

Comparison table added for revision; the first three rows follow the notes.

Data types

#	Data type	Meaning
1	**Number**	Numerical value; Number (n, d) — n = total digits, d = digits after the decimal point
2	**Char**	Max size 255
3	**Varchar**	Max size 2000 characters
4	**Date**	Various date formats (e.g. DOB)
5	**Long**	Used to store variable-length data up to 2 GB
6	**Raw**	2000 bytes — store binary file
7	**Long raw**	2 GB
8	**Time**	Store time

Structure of an SQL command



Select Name from Student where Rollno = 109 — keywords, identifiers and clauses.

- **Keywords**: Select, from, where.
- **Identifiers**: Name, Student, Rollno (names chosen by the user).
- **Clauses**: “from Student” and “where Rollno = 109”.

SQL logical operators

ALL, AND, ANY, BETWEEN, EXISTS, IN, LIKE, NOT, OR

Commands with syntax and examples

(1) Create

```
Create database Student;
use Student;

Create table TableName (CN-1 DT-1, CN-2 DT-2, ... CN-N DT-N);

Create table Student (
   Rollno Number(6),
   Name   varchar2(25),
   class  Number(3),
   DOB    Date,
   fee    Number(6,3),
   Primary key (Rollno)
);
```

(2) Describe (DESC) — display all columns with constraints

```
Describe Student;
```

Name	Null?	Datatype
Rollno	NOT NULL	Number(6)
Name		Varchar2(25)
Class		Number(3)
DOB		Date
Fee		Number(6,3)

(3) Alter — change / modify table

```
Alter table Student ADD (Address varchar(25), Phone Number(12));

Alter table Student modify (Phone Number(7));

Alter table Student DROP (column_name);
```

Syntax pattern: ADD / MODIFY (column-name, data-type).

(4) Drop

```
DROP table Student;
```

(5) Insert — single row or multiple rows

```
Insert into Student (Id, Name, Age, Phone) values
   (103, 'Seeta', 19, 1234567891),
   (104, 'Ram',   21, 2468974456);
```

(6) Update

- Table update (if no WHERE condition); every row updated.
- Selected rows (if WHERE condition).

```
Update Student Set Name = 'Malti' where ID = 103;
```

(7) Delete

- Delete an existing row.
- All records deleted (if no WHERE clause).

```
DELETE from Student where ID = 101;
```

The notes write “DELETE Student where ID = 101”; the standard form is DELETE FROM Student WHERE ID = 101.

(8) Select

- Retrieve data from the DB.

```
Select * from table_Name;
```

Some queries

Example: Order by (ascending by default)

```
Select ID, Name, Age from Student where class = 11 order by Name;
```

Example: Group by with AS (alias)

```
Select Age, Count(*) As Number from Student Group by Age;
```

The alias creates a **virtual column** for the group; use the **AS** keyword.

Example: Group by … Having

```
Select Age, Sum(fee) As Amount from Student
Group by Age having Sum(fee) > 20000;
```

HAVING filters the groups after grouping (WHERE filters rows before grouping).

Example: DISTINCT

```
Select DISTINCT Name, Age, fees from Student order by Name;
```

DISTINCT eliminates duplicate records. (The notes add “applies on first”; in standard SQL it applies to the whole combination of selected columns.)

SQL functions

- Result generation by **manipulating available data items**.
- Many built-in functions, categorised into **group functions** and **scalar functions**.

(1) Group function

- A function that acts on a **set of values**.
- Can take an entire column of data as its argument and produce a **single value** as result.
- e.g. **AVG(), COUNT(), SUM(), MIN(), MAX()**.

(2) Scalar function

- Works on **only one value**.

Category	Functions
Numeric	ABS(), SQRT(), POWER(), ROUND()
String	UPPER(), LOWER(), LENGTH(), LTRIM(), RTRIM(), SUBSTR()
Date	LAST_DAY(), NEXT_DAY(), MONTHS_BETWEEN()
Conversion	TO_NUMBER, TO_CHAR, TO_DATE

12. SQL Constraints

- Rules enforced on the data columns of a table.
- Ensure the **accuracy and reliability** of data in the table.
- 2 types: ① **column level** ② **table level**.

(A) NOT NULL constraint

- By default a column can hold NULL values.
- When a column is defined NOT NULL, it becomes **mandatory**.

```
Create table Product (
   PID     Number(4)   Not Null,
   PName   varchar(25) Not Null,
   Company varchar(25),
   Price   Number(6,3)
);
```

(B) UNIQUE key constraint

- This constraint ensures that **no two rows have the same value** in the specified column.

```
Create table product (
   PID   Number(4) Not Null Unique,
   PName varchar(25)
);
```

(C) PRIMARY KEY constraint

- To identify a record / row **uniquely**.
- By default this column is **Not Null and Unique**.

```
Create table product (
   PID   Number(4) Primary key,
   PName varchar(25) Not Null
);
```

(D) FOREIGN KEY constraint

- Used to provide a **relationship between two tables**, that is why it is referred to as the **reference key**.
- A foreign key is a primary key in another table.

```
Create table product (
   PID     Number(4) Primary key,
   PName   varchar(25),
   Company varchar(25) REFERENCES Employee(Company)
);
```

(E) DEFAULT constraint

- To set a **default value** if no value is passed through the INSERT command.

```
Create table product (
   PID   Number(4) Primary key,
   Price Number(6,3) DEFAULT 100000.000
);
```

(F) CHECK constraint

- To apply a **TRUE / FALSE condition**.
- Only if the condition is true can we insert the value in that column.

```
Create table product (
   PID   Number(3) Primary key,
   PName varchar(25) CHECK (PName LIKE 'P%')
);
```

(G) Table constraint / column constraint

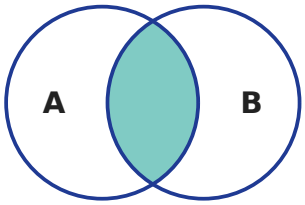
- A constraint applied on a **single column** is called a **column constraint**.
- A constraint applied on a **group of columns** is called a **table constraint**.
- Column constraints are written along with each column definition.
- Table constraints are written **after all the column constraints** are defined, at the end of the CREATE TABLE statement.

```
Create table product (
   PID     Number(4) not null,        -- column constraint
   Company varchar(25),
   PName   varchar(25) not null,      -- column constraint
   Primary key (PID, PName)           -- table constraint
);
```

13. SQL JOIN

- Used to **combine two or more tables**.
- Types:

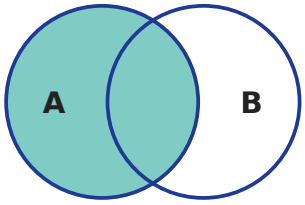
(A) Inner join



Returns only the rows that match in **both** tables (A ∩ B — the shaded overlap).

```
Select table1.col1, table1.col2
from table1
INNER JOIN table2
ON table1.matching_column = table2.matching_column;
```

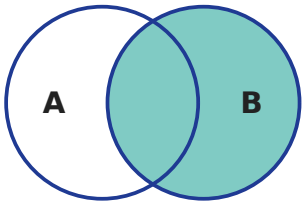
(B) Left join



All rows of the **left** table (A) plus the matching rows of B.

```
Select column_names from table1
left join table2 on table1.col = table2.col;
```

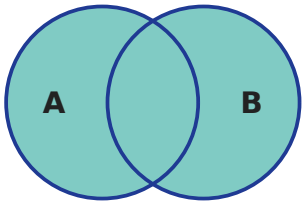
(C) Right join



All rows of the **right** table (B) plus the matching rows of A.

```
Select column from table1
RIGHT JOIN table2 ON table1.col = table2.col;
```

(D) Full join



All rows of **both** tables (A ∪ B), matched where possible.

```
Select column from table1 full join
table2 ON table1.col1 = table2.col;
```

(E) Natural join

- Connects two tables on the basis of their **common column**.
- It returns rows by matching the values in those common columns.

> **Stray note on the page**
> A line at the bottom of the notes page says: when inserting data into any data table, some business rules are followed — these are called **data constraints** (see chapter 12).

14. Serializability

- Describes the **execution schema** of DB transactions.
- It ensures that even if transactions are interleaved, their final result is the same as if the transactions were executed one at a time in some serial order.

Types

(A) Conflict serializability

- When concurrently running transactions can be **rearranged** so that their execution sequence becomes equivalent to some serial order, the schedule is called conflict serializable.

(B) View serializability

- When concurrently running transactions are **already equivalent to a serial order** and there is no need to rearrange them.

15. Two-Phase Locking (2PL)

- A **concurrency control protocol** that ensures consistency during the execution of database transactions.
- Two phases:

(A) Growing phase

- In this phase the transaction **acquires locks** on all required resources.
- It can obtain as many locks as it needs in this phase, but **cannot release any lock yet**.

(B) Shrinking phase

- A transaction **releases the locks** that were allotted to it.
- Once a transaction enters this phase, it **cannot obtain any new locks**.

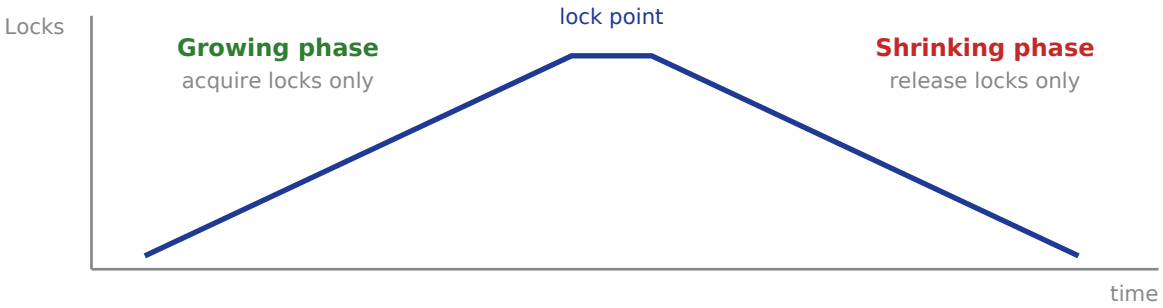


Illustration (added): number of locks held grows, reaches the lock point, then shrinks.

Advantages of 2-phase locking

- **Deadlock avoidance** (as written in the notes — this holds fully only for conservative 2PL below; basic 2PL can still deadlock).
- **Ensuring serializability** (consistency of data in the DB is not violated).

Variations of two-phase locking

Variation	Rule
(a) **Strict 2PL**	Releases its **exclusive** locks only when it finishes.
(b) **Rigorous 2PL**	A transaction can release **none** of its locks — neither shared nor exclusive — until it finishes.
(c) **Conservative 2PL**	A transaction acquires **all the locks it will need before starting execution**, so the probability of deadlock is completely removed.

16. Timestamp Concurrency Control

- A **transaction ordering** method.
- When multiple transactions are ready to execute, each is given a **timestamp (T)**, which is used to decide the order of the transactions.
- The execution priority of each transaction is determined by it.

17. NoSQL

- “Not in SQL” (non-relational database).
- Stores the data **without a structured form**.
- Data is stored by **scaling** it.
- Types of scaling:

(A) Horizontal scaling

- Adding **new server machines** alongside the existing servers.

(B) Vertical scaling

- Adding **more hardware / storage** (RAM, hard disk etc.) to an existing server machine.

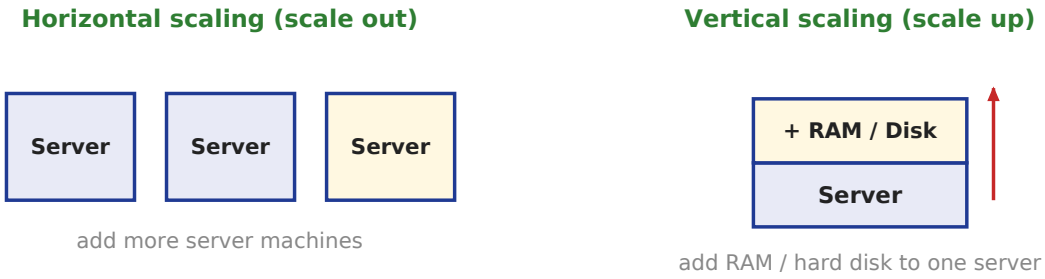


Illustration (added): horizontal vs vertical scaling.

Types of NoSQL databases

(A) Document based

- Data stored in **JSON** format.
- e.g. MongoDB, CouchDB, Firebase.

(B) Key based (key-value)

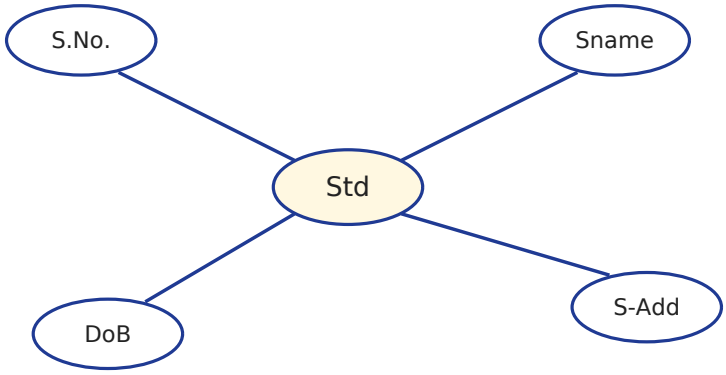
- Data stored in **key–value pairs**.
- e.g. Amazon DB (DynamoDB), Redis DB, Memcached.

(C) Wide / column based

- Database like an RDBMS, but data is stored in **columns instead of rows**.
- e.g. Apache Cassandra, HBase, Google Bigtable.
- Used in Netflix, Facebook etc.

(D) Graph based

- Data stored in the form of **nodes and edges**.
- e.g. Amazon Neptune, ArangoDB, Neo4j.



Graph database: node “Std” connected by edges to S.No., Sname, S-Add and DoB.

Comparison of NoSQL types

Feature	Document based	Key-value based	Column based	Graph based
Data model	JSON	Key–value pair	Column instead of row	Nodes & edges
Best use	Semi-structured data	Fast look-up & caching	Analytics & big data	Relationship-heavy data
Query performance	Moderate	Fast	High for analytics	Optimised for relationships
Schema	Flexible	Dynamic	Semi-structured	Schema-less
Scalability	Horizontal	High, horizontal	Highly scalable	Scales with relationships

18. ACID Properties

(a) Atomicity — "either all or none"

- Either the transaction executes **completely**, or **no part** of it will execute.

(b) Consistency

- The **total amount before a transaction starts and after it ends must remain the same**.

Example: Transfer of 500 from A to B

Step	A	B	Total
Start	1000	4000	5000
A = A − 500	500	4000	—
B = B + 500	500	4500	5000

The total is 5000 before and after → the database stays consistent.

(c) Isolation

- Parallel (concurrent) transactions should be executed in such a way that their effect is **equivalent to some serial order**, so that no inconsistency occurs in the data.

(d) Durability

- Once changes made by a transaction are **committed**, they must become **permanent** in the database, so that even after any hardware or software failure, no committed data is lost.

Appendix — Clarifications made while typing the notes

The handwritten pages were typed as faithfully as possible. These places were clarified, corrected or marked as added:

Topic	In the handwritten notes	Here
Data independence	Heading says "Data Dependence" (content describes independence)	Titled data independence
Database in "Some points"	"Collection of fields"	Kept, with a note: strictly a collection of related files / tables
ER diagram attributes	Dashed / solid ellipses hard to tell apart	Duration drawn as derived (end − start)
Super-key count, Q2	$2^3 + 2^3 + 2^2$ written	$2^3 + 2^3 - 2^2 = 12$ (matches the formula and the answer)
Codd's rules	Rules numbered (1)–(13) with a crossed-out duplicate "Rule Seven"	Listed as Rule 0 – Rule 12

Topic	In the handwritten notes	Here
2NF Q1	Candidate key line unclear	Closure check added: EC is a key; C → F and E → A are partial
2NF Q2 / 3NF Q1 / BCNF	Candidate keys as written	Copied unchanged; reasoning explained
4NF sample table	Small, slightly illegible	Reconstructed as closely as possible
Non-trivial FD	“x ∩ y ≠ 0”	X ∩ Y = ∅ (matches the Sid → S_Name example)
DELETE syntax	DELETE Student where …	Standard DELETE FROM … shown
DISTINCT	“applies on first”	Standard behaviour noted
2PL advantage	“Deadlock avoidance”	Qualified: fully true for conservative 2PL
Added for revision	—	DROP / TRUNCATE / DELETE table, many-to-many note, lock-point graph, scaling picture, normal-form summary table